

"Delivery Efficiency" Kick-off — Coverage Map

What this is. An item-by-item mapping of the *Delivery Efficiency* internal project (kick-off deck, 11 Jun 2026 — sponsor: Hrvoje Gorajščan, PM: Ivana Fazinić, runtime 15.06.–15.12.2026) against capability that **already exists or is in active build** in two assets:

Asset	What it is	Status today
Spec-First AI Development Framework — <code>gitlab.qagency.io/ai/q-ai-agents/spec-first-framework</code>	The standardized, gated, AI-assisted SDLC methodology: 6-step workflow, 11 mandatory artifact templates, 16 skills, hard human-approval gates, TDD mandate, bug/change-request tracks, CLI installer	v1.2.0, shipped — and under heavy testing right now. Installable on any project today (<code>spec-first init</code>); works in Cursor / Claude Code / OpenCode / Codex; being hardened through experimentation + testing before wider rollout
AIOps (Agency OS / AI Ops Hub) — this repo, north-star in <code>product-vision.md</code> , management brief at <code>/pitch</code>	The AIOps platform — an agent-agnostic control plane for <i>all kinds</i> of agents. Its first stream, and our biggest bet right now, is an SDLC-focused agentic system for human-agent pods : accountable humans, unified gates, deterministic (zero-LLM) validators, append-only audit ledger, per-delivery-unit economics, LAUNCH / RUN / MONITOR — the convergence target for <i>all</i> delivery, agent pods and vanilla Spec-First teams alike	BA agent live (federated), Knowledge agent operating, SA and Developer agents done or nearing completion, audit ledger real (dashboard-owned Supabase). Full product experience built demo-grade (mock-first as deliverable #0); productization staged on the agent contract

Project context (owner statement, 2026-06-12). These assets sit under the company's **only two HIGH-priority projects**. The SDLC workflow (Spec-First) and the autonomous agents — **BA, Knowledge Base, SA, Developer, all done or nearing completion** — are deliverables of **AI Resilience**; **AIOps (Agency OS)** is spinning off from AI Resilience as its own HIGH-priority project — and it is scoped deliberately wide: **AIOps is an agent-agnostic platform meant to operate *all kinds* of agents** (the SDLC pod agents today; a **PM autonomous agent** and others planned), and **its first stream — our biggest bet right now — is the SDLC-focused agentic system for human-agent pods**. The deck's own *Strategic Synergies* slide concedes this: it names **AI Resilience** as the synergy partner for "AI-Assisted Workflows & Quality Gates" — i.e., it already acknowledges that this scope lives inside an existing HIGH-priority charter. The direction of travel is set the same way: **most future client engagements will be steered to human-agent pods, and AIOps is the full platform for that delivery model** — including for teams that deliver with the vanilla Spec-First workflow in Cursor / Claude Code, who fold into the platform to get its gates, approvals, ledger, and visibility. One delivery surface; agent-run and human-run work governed identically under it. None of this is theoretical: the framework has **already been used on Harvn and on internal projects, and is introduced to TPG; weekly hands-on AI workshops** are running to train delivery teams on it; and the **SDLC workflow is under heavy testing right now** — a deliberate hardening track of experimentation and testing to make it bulletproof before it is rolled out to the rest of the projects.

Bottom line. Every one of the six project goals and five of the six scope packages in the kick-off deck map to capability that is shipped, live, or in staged build across these assets. The genuinely uncovered remainder is **organizational**, not technical: adoption mandate, training, and the behavioral "attitude challenge" the deck itself classifies as only indirectly addressable. Recommendation at the end: converge the initiatives — Delivery Efficiency as the **rollout & enforcement arm** over this tooling, not a parallel six-month build of the same machinery.

1. Scoreboard

Deck item	Coverage	Covered by
Goal: Technical Framework	✔ Covered — shipped & field-used	Spec-First v1.2.0: used on Harvn + internal projects, introduced to TPG (+ agent contract v0.5 for automation)
Goal: AI-Assisted Workflows	✔ Covered — shipped / near-complete	Spec-First (16 AI skills); AI Resilience agent fleet (BA, Knowledge, SA, Dev done or near; QA/DevOps staged)
Goal: Delivery Visibility & Predictability	✔ Covered (visibility shipped/demo-grade; predictive = named roadmap)	AIOps MONITOR; per-ticket economics; framework workflow-state files
Goal: Shippable Product (MVP mindset)	✔ Covered, shipped	Spec-First per-task verification + TDD; Agency OS blueprints & pilot plan
Goal: Project Audits	✔ Covered	Spec-First <code>/validate</code> + artifact trail; Agency OS append-only audit ledger + Compliance
Goal: Quality Gates & COE presence	✔ Covered, shipped	Spec-First hard gates (4 human + 2 mechanical); Agency OS deterministic validator floor + autonomy ladder
Scope: Project Initiation	● Theirs (org ritual)	— (we supply <code>/pitch</code> , product-vision, demo as input material)
Scope: AI in PMO	● Direct overlap with a planned build	Planned PM autonomous agent (an AIOps agent — PMO automation <i>is</i> an agent for us); AIOps MONITOR reports/digests; predictive tracking on the roadmap
Scope: Technical Framework	✔ Covered, shipped	Spec-First (see Goal 1)
Scope: Project Audit Mechanism	✔ Covered	See Goal 5; audit = artifact completeness + ledger evidence
Scope: Internal Projects Valorisation	✔ Covered — it is literally our adoption strategy	Agency OS internal-first track (owner call 2026-06-10); both repos are internal projects run with this rigor
Scope: Shippable Product Mindset	✔ Covered	Same as Goal 4

2. Their problem statement → what already answers it

The deck's *Project Background* (slide 5) names four problems. Each one is the design center of something we have already built.

2.1 "Inconsistent Delivery Quality — outcomes depend on individual contributors rather than a standardized model"

- **Spec-First removes the individual-dependence by construction.** Every feature on every project moves through the same six gated steps — `CONSTITUTION → SPEC → DESIGN → (UIX) → TASKS → Implementation → Review` — producing the same artifacts from the same templates, with a hard gate between each step (next step refuses to start unless the

previous artifact is `APPROVED`). The output quality is a property of the **process**, not the person. (`.framework/steps/`, `.framework/templates/`, 16 skills.)

- **Agency OS makes the floor deterministic.** The zero-LLM structural validators (8 on BA specs live today; SA design matrix D1–D7 and QA matrix Q1–Q7 defined) pass or fail on structure alone — *"DETERMINISTIC — NO MODEL IN THE LOOP."* No individual judgment in the floor check, by design (product-vision.md §1, moat pillar 5; `/governance`).
- **Their specific pain — greenfield & fixed-price projects** — is directly addressed twice: greenfield is Spec-First's home turf (CONSTITUTION + spec-first flow from file zero; brownfield has the `legacy_ai_analyser` companion), and **fixed-price scope protection is a built-in gate**: the `/change` workflow's hard *bug-vs-change-request classification check*, and on the platform side the SOW living in the Knowledge agent's shared context so *"bug or change request?" becomes a lookup, not a negotiation* (product-vision §1).

2.2 "Limited Technical Oversight — late identification of architectural debt, inconsistent implementation practices"

- **Architecture is gated before a line of code exists:** Gate 2 — Tech Lead approves `DESIGN.md` — blocks task breakdown and implementation. Architectural debt is caught at the design gate, not in month three. (`/design`, step-02.)
- **Implementation practice is enforced, not advised:** TDD iron law (RED→GREEN→REFACTOR per task, "code before test → delete it"), per-task verification commands, a blocking 6-point verification checklist (tests green, lint clean, coverage met, scope integrity), and a code review that is **forbidden from approving without reading the code and running the tests** (code-review skill "Iron Law", Phases 0–3, max-3 auto-fix attempts).
- **Agency OS adds fleet-level oversight:** root-cause reject (a gate failure is attributed to the *causing* stage and re-runs forward), rerun-count economics per stage, incidents with recovery controls, and the `/governance` validator panel.

2.3 "Governance Challenges — COE has limited capacity and authority; no assurance that all mandatory artefacts are being produced"

This sentence describes, almost verbatim, the two problems our stack was built to kill:

- **"No assurance artefacts are produced" → artifacts are structurally mandatory.** In Spec-First you *cannot reach* implementation without `SPEC.md`, `DESIGN.md`, `TASKS.md` all `APPROVED`; you cannot finish without `IMPLEMENTATION-SUMMARY.md` and `REVIEW.md`. `/validate` audits framework + artifact integrity in one command. Assurance stops being a person checking — it's the workflow refusing to proceed.
- **"COE has limited capacity" → the deterministic validator floor + autonomy ladder is the scaling answer.** Zero-LLM checks run on every artifact at zero marginal human cost; humans review where the ladder says review (L0 review-all → L3 auto-clear), and **promotion up the ladder is earned by deterministic validator streaks, granted by an accountable human, written to the ledger** (product-vision §1, "earned autonomy"). COE presence becomes policy, not headcount.
- **Evidence for governance:** every gate decision (approve / reject / override, with typed reason on reject/override) lands in the **append-only audit ledger** the dashboard owns (Supabase `audit_log`, survives agent resets) — `/compliance`, architecture.md. This is also our EU AI Act Article 26 deployer story.

2.4 "Accountability & Roles — roles and responsibilities not clearly defined or enforced"

- **Spec-First defines the role-gate matrix explicitly:** PO approves SPEC (Gate 1), Tech Lead approves DESIGN and TASKS (Gates 2–3), Reviewer issues the review verdict (Gate 4), Developer owns the verification gate. No gate passes without its named role acting.

- **Agency OS makes accountability a first-class object:** the accountability matrix — **one accountable human per agent, an empty column rendered as uncovered risk** — plus role-scoped gate permissions (QA Lead clears QA gates only, etc.), capacity & coverage hours, audited reassignment with one-tap acceptance, and human-decision attribution in the ledger (`/pod` , `src/mock/roles.ts` , `src/mock/humans.ts`).
- The deck's "*low proactivity in delivery teams*": the platform's answer is **proactive agents in human-native channels** — the pod reaches people in Slack with the decision one deep-link away, instead of waiting for someone to open a dashboard (moat pillar 3).

The deck's own split (slide 6) — *process gaps directly addressable, attitude challenge only indirectly* — matches this map exactly: **the process half is what our tooling already does; the attitude half is org work that no new tooling project would solve either**. That residual is the legitimate core of their project (see §6).

3. Their six goals, one by one

Goal 1 — "Technical Framework: a clear, standardized technical framework for consistency"

Covered — shipped. This deliverable exists, is versioned, and is installed per project in one command.

What the goal needs	What exists	Where
A standard every project follows	Spec-First v1.2.0: 6-step gated workflow, identical on every project	<code>README.md</code> , <code>.framework/steps/</code>
Project-level standards baseline	<code>CONSTITUTION.md</code> per project (tech stack, coding/testing/API/security standards, quality-gate commands) — every downstream step validates against it	<code>/constitute</code> , <code>CONSTITUTION.template.md</code>
Standardized artifacts	11 templates (SPEC, DESIGN, UIX-SPEC, TASKS, REVIEW, BUG, CHANGE-PROPOSAL, ...)	<code>.framework/templates/</code>
Distribution & evolution	<code>spec-first init</code> / <code>spec-first update</code> (preserves project specs & constitution), semver, plugin for Cursor / Claude Code / OpenCode / Codex	<code>spec-first.sh</code>
A standard for automation consistency	Agency OS agent contract v0.5 (shared envelope + per-role facet), A2A-aligned Agent Card, agent SDK — any agent/tool plugs into the same lifecycle	<code>src/contract/</code> , <code>agent-sdk-brief.md</code>
Proof it works on real delivery	Used on Harvn and internal projects; introduced to TPG. Weekly hands-on AI workshops train delivery teams on it; the workflow is under heavy testing right now — a deliberate hardening track (experimentation + testing) that gates broader rollout	field usage per product owner, 2026-06-12

A new "technical framework" work package would re-produce this repo. The open work is **adoption** (which projects, by when, who enforces `spec-first init`), not creation — and adoption's end-state is on-platform: Spec-First teams in Cursor / Claude Code fold into Agency OS so their gates and artifacts carry the same ledger evidence as agent-run pods.

Goal 2 — "AI-Assisted Workflows: leveraging AI to optimize delivery speed and accuracy where appropriate"

Covered — shipped (framework), live + staged (platform).

- The framework is an AI-assisted workflow end to end: 16 skills / slash commands (`/flow` , `/specify` , `/design` , `/uix` , `/tasks` , `/implement` with subagent-driven multi-task execution, `/review` , `/adversarial` , `/bug*` , `/change` , `/debug`) — each phase AI-drafted, human-gated. "Where appropriate" is answered by the philosophy line the framework enforces: *AI proposes, human approves* — no auto-bypass at any gate.
- The same SDLC runs as an **agent fleet**, built under AI Resilience: BA agent **live in production federation today** (spec drafting, EARS criteria, clarification gates), Knowledge agent operating and mandatory in every pod, **SA and Developer agents done or nearing completion**, QA / DevOps staged on the same contract (product-vision.md §1; 7-role catalog in `src/mock/chain.ts`).

Goal 3 — "Delivery Visibility and Predictability: improving real-time insights into overall project progress"

Covered for visibility; predictive analytics is an honest, named roadmap item.

- **Real-time progress:** Agency OS MONITOR — Overview KPI strip (in-flight / gated / ready), pipeline board over the artifact lifecycle (`backlog` → `in_progress` → `waiting` → `ready` → `approved` → `delivered`), agent status grid, live gate queue, SSE event stream from the live BA agent (`/` , `/pipeline` , `/approvals` , `/flow`).
- **Reporting without asking a PM:** weekly client-clean status report (items delivered, gates cleared, cycle time, cost — exportable), read-only Exec Digest for sponsors, SLA target-vs-actual with breach history (`/reports`), notification digests.
- **Delivery telemetry:** per-ticket economics — cost by stage (cloud vs. local inference), human-review minutes, rerun penalty, cycle time (`/economics` , `src/mock/economics.ts`); framework `.workflow-state.md` files add per-feature progress, implementation-attempt counts, and `jiraTicket` / `sowRef` linkage. Session-level cost/usage telemetry is owned by the AIOps platform itself.
- **Predictability:** cycle-time, gate-latency, and rerun economics exist now (`/economics` , `src/mock/economics.ts`); *predictive/forecasting* features (auto-clear previews, drift→incident prediction) sit on the platform's deferred register — tracked, not vapor.

Goal 4 — "Shippable Product: delivering real value early with an MVP mindset"

Covered — it's enforced mechanically, not aspirationally.

- Spec-First makes every increment shippable by gate: per-task TDD with task-scoped verify commands, the blocking verification checklist (suite green, lint clean, coverage met) **before** review, git-worktree isolation and a finishing-branch skill that merges/PRs only after an `APPROVED` review verdict. A sprint outcome that doesn't pass the gates doesn't exist.
- Agency OS productizes the same mindset: blueprints launch a working pod in minutes; activation metric is **time-to-first-approved-artifact < 24h**; the pilot plan converts a 4-week pilot into annual economics (`src/mock/blueprints.ts` , `src/mock/pilot.ts`). Culturally: the platform's own deliverable #0 is the working mock — value first, runtime second.

Goal 5 — "Project Audits: introducing mechanisms to ensure compliance and quality"

Covered — with evidence that survives.

Audit need	Mechanism	Where
Are mandatory artifacts present & well-formed?	<code>/validate</code> integrity check; hard gates make missing artifacts impossible to skip past; deterministic validators score structure per artifact	framework; <code>/governance</code>
Who decided what, when, why?	Append-only <code>audit_log</code> (dashboard-owned Supabase): every gate approve/reject/override with typed reason, pause/resume, policy change, reassignment — survives agent resets	<code>/compliance</code> , <code>src/lib/db/audit.server.ts</code>
Requirements → code → tests trail	Spec ID → DESIGN traceability table → per-task AC mapping → AC-named tests → REVIEW.md Phase-2 evidence table; Bug History & Amendment History self-update on the spec	framework templates; <code>/traceability</code>
Regulatory framing	EU AI Act Art. 26 deployer mapping (oversight = accountability matrix, logs = ledger, incidents = built-in) — already a panel on <code>/compliance</code>	product-vision §1

An audit mechanism designed from scratch would have to invent exactly this. The cheaper move: **define "auditable project" as "project on the framework + ledger"** and point COE audits at `/validate` output and the Compliance surface.

Goal 6 — "Quality Gates and COE presence: strengthening checkpoints throughout the project lifecycle"

Covered — gates exist at both the methodology and platform layer.

- **Methodology layer (shipped):** 4 blocking human gates (SPEC → PO; DESIGN, TASKS → Tech Lead; REVIEW verdict → Reviewer) + the mechanical verification gate + per-task validation, with capped retry loops (3) and explicit BLOCKED states.
- **Platform layer (live + demo-grade):** unified gates queue with a client-grade review surface (rendered SPEC, EARS criteria, validator results on the decision panel), reject requires a typed reason, root-cause reject re-runs downstream stages, and every decision is ledger-audited (`/approvals`, `src/components/gates/`). These gates are not agent-only: the convergence end-state folds vanilla Spec-First teams into this same queue — a human-produced SPEC clears the same gate, with the same ledger evidence, as an agent-produced one.
- **"COE presence" without COE headcount:** the deterministic validator floor runs on every artifact, and the autonomy ladder concentrates scarce human review exactly where streak evidence says it's still needed — *autonomy you can audit your way into* (product-vision §1).

4. Their "Strategic Synergies" slide — we are the synergy

The deck names three synergies; each one lands on something in this stack:

Deck synergy	Reality
Delivery Visibility ↔ QPT Dashboard ("automated tracking")	The tracking data already exists in machine-readable form: per-ticket economics, framework workflow-state files, AIOps MONITOR + audit ledger. QPT should consume these feeds , not commission a second collection pipeline. AIOps is deliberately tracker-friendly: tickets stay in Jira/Teamwork, status written back, the board is the doorbell.
AI-Assisted Workflows & Quality Gates ↔ AI Resilience ("resilient human-AI delivery")	This is not a synergy to be arranged — it is an acknowledgment of ownership. The SDLC workflow and the autonomous agents (BA, Knowledge Base, SA, Developer — done or nearing completion) are AI Resilience deliverables , and the gates / validator floor / AgentOps harness belong to AIOps, its spin-off — together the company's only two HIGH-priority projects. "Resilient human-AI delivery" is already the product's one-line positioning: <i>not an autonomous coder you must trust — a governed agent–human pipeline you can prove</i> . This deck goal is, in effect, a commitment to adopt those projects' output.
Shippable Product ↔ Account Growth	Already the platform's commercial logic: internal-first adoption where every Q-led project doubles as a live client reference , the trust bump wins SDLC deals, pilot→annual conversion sheet, exportable client status reports (product-vision §1).

5. Their preliminary scope, package by package

Their work package	Status against our stack
Project Initiation (setup, stakeholder mapping, resource planning)	● Org ritual — legitimately theirs. Inputs we already have ready: <code>/pitch</code> (management brief, print-to-PDF), product-vision.md, the live demo, this document.
AI in PMO (predictive tracking, automated reporting)	● Direct overlap with a planned build. In our terms "AI in PMO" <i>is</i> a PM autonomous agent — which we are already planning — plus automated reporting (AIOps MONITOR weekly reports / digests / notifications, demo-grade now, productizing under AIOps) and predictive tracking (roadmap). Because AIOps is agent-agnostic, the PM agent is simply another agent it governs. A separate PMO-AI tool here would duplicate a named AIOps agent — fund the requirements into it instead.
Technical Framework (standardized SDLC practices, baseline metrics)	✅ Spec-First v1.2.0 is this package, shipped. Baseline metrics: live tier (cost-per-approved-spec, time-to-approved-spec from the BA agent), per-ticket economics model, pilot week-1 baseline capture, ROI assumptions with provenance ("your numbers, not ours").
Project Audit Mechanism (structured reviews for compliance & technical quality)	✅ See Goal 5. Define the audit checklist as: artifacts complete (<code>/validate</code>), gates passed with reasons (ledger), validator pass-rates (<code>/governance</code>), review verdicts (REVIEW.md). Structured review of <i>content</i> exists too: <code>/adversarial</code> (≥10-issue skeptical review of any document) and the 4-phase code review.
Internal Projects Valorisation (same rigor for internal as client work)	✅ This is, word for word, the platform's adoption strategy: internal-first, sellable second (owner call 2026-06-10) — Q-led projects run on the pods first, every one doubles as a live reference. Already exercised in practice: the framework runs on internal projects today (alongside Harvn, with TPG introduced), and the framework and platform are themselves internal projects run under this rigor (specs, gates, audit trail in-repo).
Shippable Product Mindset (incremental value, production-ready sprint outcomes)	✅ See Goal 4 — enforced by gates, not posters.

6. What their deck contains that we genuinely do *not* cover

Being honest about the residual is what makes the rest of this document credible:

1. **The attitude / behavioral challenge** (slide 6) — proactivity, ownership culture, performance management. Tooling shapes behavior (gates force artifacts; Slack-native agents lower the friction of acting) but doesn't change attitudes. *This is the strongest unique mandate their project has.*
2. **COE authority & adoption mandate** — deciding that projects **must** run `spec-first init`, that audits use the ledger, and making that stick org-politically. Our stack makes compliance *checkable in one command*; it cannot make it *mandatory*. That's a management decision their project can own.
3. **Training & enablement at scale** — partially closed already: **weekly hands-on AI workshops are running** on the framework. The residual is formalizing curriculum, coverage, and attendance across all delivery teams (materials exist: framework README/PHILOSOPHY, ONBOARDING in the agent monorepo, `/pitch`, the demo script, the workshop series).
4. **Audit cadence for legacy projects not on the framework** — partially bridged (the `legacy_ai_analyser` companion generates a brownfield CONSTITUTION), but a manual audit checklist for never-migrated projects would be new work.
5. **True predictive analytics & the PM agent** — forecasting (not just live metrics) is on our roadmap, not shipped, and the **PM autonomous agent is planned, not built**. These are real gaps *today* — but they are named AIOps roadmap items, so the fix is to spec them into the AIOps backlog, not to start a parallel PMO-AI build.
6. Known honest gaps inside our own stack (already tracked): ledger actor attribution is "when-only" until real auth lands; QA and DevOps/Release agents are still staged behind the done-or-near BA / Knowledge / SA / Dev set; most dashboard surfaces are demo-grade ahead of productization. None of these are things a parallel project would solve faster than the two HIGH-priority build tracks that own them.

7. Recommendation — converge, don't duplicate

The deck describes a 6-month internal project to create what is substantially **already created and partly in production**. The priority math makes the case by itself: the overlapping scope is the charter of the company's **only two HIGH-priority projects — AI Resilience and its AIOps spin-off** — and the deck's own synergy slide concedes the dependency on AI Resilience. Rebuilding that scope inside an internal-category project would invert the company's own prioritization. The operational risk if it proceeds independently: two SDLC standards, two reporting pipelines, two audit mechanisms, and a credibility cost for both efforts. Proposed convergence:

1. **Before their 15.06 start: a working session** (us + Ivana + Hrvoje) — live walk-through of Spec-First (`/flow` on a real feature) and AIOps / Agency OS (`/pitch` + demo). Goal: agree on the build/adopt split below, and route any residual build items into the AI Resilience / AIOps backlogs rather than opening a third build track.
2. **Technical Framework deliverable → adopt Spec-First v1.2.0**. Their project owns the rollout plan: which projects, in what order, CONSTITUTION authoring per project, COE as the enforcement authority. Framework gaps they discover become issues on the framework repo — one standard, jointly hardened. Adopting the framework is step one of landing on the platform, not an alternative to it. And rollout is not starting from zero: **already used on Harvn and internal projects, introduced to TPG, taught in weekly hands-on workshops, with a deliberate hardening track (experimentation + testing) gating adoption on the rest of the projects** — their plan should extend this sequence, not restart it.
3. **Delivery-model end-state → one platform**. Per owner direction, most future client engagements are steered to **human-agent pods, with AIOps as the full platform for that model**. Teams delivering with vanilla Spec-First in Cursor / Claude Code **fold into the same platform**, so gates, approvals, audit evidence, and client reporting are identical regardless of whether a human or an agent produced the artifact — no second operating model for "manual" projects.

4. **AI in PMO deliverable → it's a planned AIOps agent, not a new tool.** We are already planning a **PM autonomous agent**; because AIOps is agent-agnostic, PMO automation lands as an agent on the platform, not a parallel system. Use AIOps MONITOR reports/digests for stakeholder reporting now; write the *PM-agent* and *predictive-tracking* requirements as specs into the AIOps backlog — build once, on data that already flows.
5. **Project Audit Mechanism deliverable → define audit = evidence we already emit.** Audit checklist: `/validate` pass, gate decisions with reasons in the ledger, validator pass-rates, REVIEW verdicts. COE audits read evidence instead of conducting archaeology.
6. **Visibility / QPT synergy → QPT consumes our feeds** (platform economics/telemetry, audit ledger, workflow state) rather than instrumenting delivery a second way.
7. **Their unique scope — own it fully:** adoption mandate, training program, behavioral/attitude work, legacy-project audit cadence, internal-project valorisation policy. That is a real, valuable project — and it's the part no tooling build would deliver.

Net effect: Delivery Efficiency keeps its goals and its timeline, drops ~4 of 6 scope packages from "build" to "adopt + enforce", and the company gets **one delivery standard on one platform with one evidence trail** — agent-run and human-run work alike — which was the deck's stated intent: *business impact, beyond code*.

Sources: "Delivery efficiency — Kick off.pptx" (11 Jun 2026, slides 1–11); spec-first-framework @ v1.2.0 (README, PHILOSOPHY, `.framework/`, 16 skills); this repo's product-vision.md, architecture.md, agent-sdk-brief.md, `/pitch`; project hierarchy, priority status, agent maturity, platform framing (agent-agnostic AIOps; SDLC first stream; planned PM agent), and field-usage status (Harvn, internal projects, TPG; weekly workshops; hardening track) per product-owner statements (12 Jun 2026). Compiled 12 Jun 2026.